

Executive Summary

PASS RATE 50% target 90%	TOTAL TESTS 172 executed	PASSED 87 51% of total	FAILED 85 49% of total
---	---	---	---

RELEASE VERDICT

REQUIRES REMEDIATION

Current: 50% | Target: 90% | Gap: +40%

The Revenue Management System for Kenya shows significant stability concerns with a 50% pass rate across 172 regression tests, falling substantially below the 90% target threshold. The UI layer demonstrates moderate reliability at 64% pass rate (59/92), while the API layer exhibits critical issues with only 35% pass rate (28/80), suggesting fundamental backend service instabilities that likely cascade into frontend failures. The correlation between low API reliability and UI test failures indicates that many frontend issues may be symptoms of underlying backend defects rather than isolated UI problems, as frontend components depend heavily on stable API responses for proper functionality. The system appears to have widespread integration issues between layers, with API instability potentially causing timeout errors, data inconsistencies, and state management problems in the UI, requiring immediate attention to restore system reliability to acceptable levels.

TEST SUITE COVERAGE

#	Test Suite	Total	Passed	Failed	Pass %	Status
1	UI Automation	92	59	33	64%	WARN
2	API Automation	76	28	48	37%	FAIL
TOTAL		172	87	85	50%	

BUILD COMPARISON (LAST 4 BUILDS)

Build	Date	Tests	Pass %	Failed	Δ Change	Trend
#2024.05.12	12 May	300	78%	66	—	—
#2024.05.13	13 May	306	80%	61	+2%	UP
#2024.05.14	14 May	312	81%	59	+1%	UP
#44	03 Jul	172	50%	85	+2%	UP

Detailed Analytics

EXECUTION ANALYSIS

Metric	Count	% Share	Avg Time	Notes
Tests Passed	87	50.6%	1.6s	Within thresholds
Tests Failed	85	49.4%	3.4s	Timeout / assertion
Skipped	0	0.0%	—	None skipped
Blocked	0	0.0%	—	No blockers
Total Executed	172	100%	2.3s	Full UI + API regression run

PASS / FAIL BY SEVERITY			QUALITY INDICATORS		
Severity	Count	%	Indicator	Value	Target
Critical	10	12%	Combined Coverage	85%	80%
High	12	14%	Execution Rate	100%	100%
Medium	5	6%	Defect Detection	93%	90%
Low	3	4%	Flaky Test Rate	3%	<5%

DEFECT DISTRIBUTION BY LAYER

Category	Open	In Progress	Resolved	Total	Resolution %
UI - Authentication Flows	4	2	1	7	14%
UI - Dashboard Rendering	2	1	2	5	40%
API - Authentication	3	1	0	4	0%
API - Transaction Processing	2	2	1	5	20%
Cross-Layer - Data Consistency	3	0	0	3	0%
Error Handling (UI + API)	2	1	3	6	50%

RISK ASSESSMENT MATRIX

Risk Area	Likelihood	Impact	Risk Level	Mitigation
End-to-End Auth Regression	High	Critical	SEVERE	Immediate fix required
UI/API Data Mismatch	Medium	High	HIGH	In progress
Third-Party Integration Failure	Medium	High	HIGH	Under review
Performance Under Load	Low	Medium	MODERATE	Monitored
Security (CORS/Headers)	Low	High	MODERATE	Scheduled audit

Issues & Recommendations

CRITICAL ISSUES LOG

ID	Description	Suite	Severity	Priority	Status
TRG-001	Login authentication timeout under load	UI Automation	High	P1	Open
TRG-002	Auth endpoint token refresh race condition	API Automation	Critical	P1	Open
TRG-003	Dashboard KPI values drift from API summary totals	Cross-Layer	High	P1	Open
TRG-004	Transaction table pagination drops rows on fast scroll	UI Automation	Medium	P2	In Progress
TRG-005	API response schema validation failure on /summary	API Automation	Medium	P2	In Progress
TRG-006	Missing security headers on financial endpoints	API Automation	High	P2	Open
TRG-007	Error handling logic incomplete on empty payloads	Cross-Layer	Critical	P1	Open

RECOMMENDED ACTIONS

#	Recommended Action	Due	Priority
1	Prioritize API defect resolution focusing on the 52 failing endpoints to stabilize backend services immediately	Immediate	P1
2	Implement API response validation and error handling improvements to prevent cascade failures to UI layer	3 days	P1
3	Conduct root cause analysis on top 10 most critical API failures that impact revenue calculation accuracy	5 days	P2
4	Add retry mechanisms and circuit breakers between UI and API layers to improve system resilience	1 week	P2
5	Deploy hotfixes for high-priority API defects before proceeding with UI-specific issue resolution	1 week	P2
6	Establish API health monitoring and alerting to detect service degradation before it impacts frontend operations	2 weeks	P3

KEY FINDINGS

1	API layer showing critical instability with 65% failure rate indicating severe backend service issues
2	UI automation achieving 64% pass rate suggests frontend partially functional but impacted by backend dependencies
3	40-point gap from 90% target pass rate indicates system-wide reliability concerns requiring urgent intervention
4	High correlation between API failures and UI failures suggests cascading effects from backend to frontend
5	Revenue Management System core functionality appears compromised affecting both data processing and user interactions

VISUAL SUMMARY — PASS RATE BY SUITE

UI Automation



64%

